Project Plan

AI-Powered Infrastructure Monitoring with Kubernetes and Cilium

Research

Bas Smit

Fontys University of Applied Sciences

Fontys
University of Applied Sciences

Contents

1 Introduction **1**
1.1 Background and Motivation . . . 1
1.2 Project Scope . . . 1
1.3 Relevance to Enterprise Infrastructure . . . 2

2 Research Questions and Project Objectives **3**
2.1 Main Research Question . . . 3
2.2 Sub-Questions . . . 3

3 Methodology and Research Framework **4**
3.1 Library Research . . . 4
3.1.1 Literature Study . . . 4
3.1.2 Available Product Analysis . . . 4
3.1.3 Best, Good, and Bad Practices . . . 4
3.1.4 IT Architecture Sketching . . . 5
3.2 Field Research . . . 5
3.2.1 Problem Analysis . . . 5
3.2.2 Domain Modelling . . . 5
3.3 Lab Research . . . 5
3.3.1 Prototyping . . . 5
3.3.2 System Test . . . 6
3.3.3 Security Test . . . 6
3.3.4 Non-Functional Test . . . 6
3.4 Showroom Research . . . 6
3.4.1 Benchmark Test . . . 6
3.4.2 Guideline Conformity Analysis . . . 6
3.5 Framework Triangulation . . . 6
3.6 Additional Frameworks . . . 6

4 Non-Functional Requirements **8**
4.1 Scalability . . . 8
4.2 Performance . . . 8
4.3 Security . . . 8
4.4 Reliability . . . 9
4.5 Observability . . . 9
4.6 Maintainability and Portability . . . 9

5 Planning, Timeline, and Expected Challenges **10**
5.1 Project Timeline . . . 10
5.2 Time Estimation . . . 11
5.3 Expected Deliverables . . . 12
5.4 Expected Challenges . . . 12
5.4.1 Training Data for AI Anomaly Detection . . . 12
5.4.2 eBPF Kernel Version Compatibility . . . 12
5.4.3 Integration Complexity . . . 12
5.4.4 Scope Creep . . . 13
5.5 Supervisor Approval . . . 13

1 Introduction

1.1. Background and Motivation

Enterprise infrastructure has become way more complex over the past decade. Kubernetes is now the go-to solution for running containers, but using it brings two ongoing challenges: *reactive monitoring* and *poor visibility into network security*.

Traditional monitoring systems rely on fixed thresholds. They're built for static, predictable workloads. But Kubernetes is anything but static — pods come and go constantly, traffic patterns change all the time, and dozens of microservices talk to each other. Fixed thresholds lead to tons of false alarms and often miss subtle issues that actually matter. Engineers get overwhelmed with alerts (alert fatigue), and problems only get noticed *after* they've already affected users. It's a reactive approach, not a proactive one.

On the security side, Kubernetes networking by default uses `iptables` rules managed by `kube-proxy`. This only works at Layer 3/4 (IP addresses and ports), gives you limited insight into what's actually happening, and becomes a bottleneck when you scale up. You can't see application-level traffic details, and you have no way to detect suspicious activity like lateral movement inside your cluster.

Cilium[1] fixes these networking problems by using eBPF (extended Berkeley Packet Filter) — a technology that lets you run safe programs inside the Linux kernel without changing the kernel itself. Cilium can enforce network policies from Layer 3 all the way up to Layer 7 (including HTTP, gRPC, and Kafka). It encrypts traffic between workloads with mutual TLS and provides detailed, real-time network flow data through its Hubble observability component.

Here's the key insight: combining AI-driven monitoring with Cilium's network security creates something powerful. Cilium's Hubble generates detailed per-flow data that's perfect for training machine learning models to detect anomalies. At the same time, when AI detects something unusual in your infrastructure metrics, it can trigger automated responses at the network layer. Together, you get infrastructure that's *self-aware* and *self-defending* — it spots problems early and responds to threats automatically.

1.2. Project Scope

This project explores how to design, build, and test an integrated system that combines AI-powered monitoring with Cilium-secured Kubernetes networking to enable proactive, automated infrastructure management.

What we'll cover:

- Understanding the current limitations of Kubernetes monitoring and networking

[1] `https://cilium.io`

- Designing an integrated architecture using Prometheus, an AI anomaly detection engine, Cilium, and Hubble
- Building a proof-of-concept on a bare-metal Kubernetes cluster
- Testing how well it works and measuring the performance impact

1.3. Relevance to Enterprise Infrastructure

Infrastructure teams are dealing with growing pressure: keep services running reliably while managing increasingly complex, distributed systems. Proactive anomaly detection helps you find and fix problems faster (reducing MTTD and MTTR), which means better uptime. Meanwhile, the shift to zero-trust networking — required by frameworks like NIST SP 800-207 and many enterprise security policies — needs the kind of identity-aware, application-level network controls that Cilium provides.

By combining these capabilities, this project fills a real gap in today's enterprise tooling. The result is a practical, open-source-based approach that infrastructure teams can actually use.

2

Research Questions and Project Objectives

2.1. Main Research Question

How can we design and build an AI-powered monitoring system that works together with Cilium-based network security to enable proactive, automated management of Kubernetes infrastructure?

2.2. Sub-Questions

To answer this, we'll break it down into five focused questions, each looking at a different piece of the puzzle:

1. **What are the main problems with traditional infrastructure monitoring and network security in Kubernetes?**
2. **Which AI and machine learning techniques work best for detecting anomalies and predicting problems in infrastructure and network metrics?**
3. **How does Cilium's eBPF-based approach improve network security and observability compared to traditional Kubernetes networking?**
4. **What kind of integration architecture do we need so the AI monitoring system can use Cilium and Hubble data to provide correlated security and performance insights?**
5. **How can we test and validate that the integrated solution actually works well, doesn't slow things down too much, and improves security?**

3

Methodology and Research Framework

This project uses the *Development Oriented Triangulation* (DOT) framework[1] to structure our research. The DOT framework breaks things down into three levels: *what* we're researching (domains), *why* we're researching it (balancing fit, expertise, overview, and certainty), and *how* we're researching it (strategies and methods). Below, we'll map out which research strategies and methods we'll use for each part of the project.

3.1. Library Research

Library research explores what's already known and what tools, theories, and guidelines can help us. It mainly looks at the *available work* domain and focuses on building *expertise*.

3.1.1. Literature Study

We'll conduct a systematic literature study covering:

- AI/ML methods for time-series anomaly detection (Isolation Forest, LSTM autoencoders, Prophet, ARIMA, and transformer-based approaches)
- eBPF internals, the Cilium architecture, and the Hubble observability layer
- Zero-trust network models (NIST SP 800-207) and how they apply to Kubernetes
- The CNCF landscape for observability and security tooling

This method primarily answers sub-questions 1, 2, and 3.

3.1.2. Available Product Analysis

We'll do a comparative analysis of existing tools in two categories:

- **Monitoring stack:** Prometheus, VictoriaMetrics, Grafana, Loki, OpenTelemetry Collector, Jaeger, and commercial alternatives (Datadog, Dynatrace, New Relic)
- **Security and networking:** Cilium, Calico, Flannel, Falco, Tetragon, and Hubble

We'll assess each tool based on open-source license, how well it works with Kubernetes, community maturity, feature coverage, and how easy it is to integrate.

3.1.3. Best, Good, and Bad Practices

We'll review these industry standards and guidelines:

[1] `https://ictresearchmethods.nl/dot-framework/`

- CIS Kubernetes Benchmark (v1.9)
- CNCF Cloud Native Security Whitepaper
- NIST SP 800-207 (Zero Trust Architecture)
- OWASP Kubernetes Security Cheat Sheet

3.1.4. IT Architecture Sketching

Using ArchiMate 3.2 notation, we'll produce architecture views based on the ArchiMate 3.2 specification, TOGAF ADM documentation, and reference architectures from CNCF projects:

- **Motivation view:** Goals, requirements, and constraints
- **Application layer view:** Monitoring components and their data flows
- **Technology layer view:** Kubernetes objects, Cilium, and the AI inference pipeline
- **Implementation and migration view:** Deployment phases

This method uses documented architectural patterns and standards from available work, addressing sub-question 4.

3.2. Field Research

Field research explores the *application domain* and focuses on making sure the solution actually *fits* its operational context.

3.2.1. Problem Analysis

We'll do a structured analysis of the monitoring and security gaps in a representative, freshly deployed Kubernetes cluster (using kubeadm with three worker nodes). The analysis will document:

- How many alerts fire per day using only threshold-based rules
- Network flows that are invisible to the default networking stack
- Incidents that would go undetected without AI-based detection

This method directly answers sub-question 1.

3.2.2. Domain Modelling

We'll create an ArchiMate domain model representing the enterprise Kubernetes infrastructure: nodes, namespaces, deployments, services, ingress controllers, storage classes, and external dependencies. This model gives us a shared vocabulary for the architecture design phase.

3.3. Lab Research

Lab research tests whether the solution actually works as intended. It focuses on *certainty*.

3.3.1. Prototyping

We'll use an iterative approach to implement and refine each major component, following official documentation and established practices:

1. Cilium installation and basic NetworkPolicy enforcement
2. Hubble Relay and Hubble UI for flow visualization
3. Prometheus + Grafana stack with custom Kubernetes dashboards
4. AI anomaly detection pipeline (training on synthetic normal-traffic baseline, inference via REST API)
5. Integration layer: Hubble flow exporter → OpenTelemetry Collector → AI feature store

Key resources include official project documentation (Cilium, Kubernetes, Prometheus), GitHub repositories with example implementations, and community best practices.

3.3.2. System Test

An end-to-end system test will verify that the full monitoring and security stack operates correctly: metrics get scraped, network flows are captured, anomaly detection triggers on injected anomalies, and alerts reach the notification channel.

3.3.3. Security Test

We'll validate Cilium network policies by trying traffic flows that should be blocked (like cross-namespace access without an explicit allow policy, or egress to an unauthorized external host). Tetragon will verify syscall-level enforcement.

3.3.4. Non-Functional Test

We'll measure these non-functional properties:

- CPU and memory overhead from Cilium/eBPF compared to a Flannel baseline
- Monitoring pipeline latency (time from metric generation to alert firing)
- AI inference throughput and model prediction latency under load

3.4. Showroom Research

3.4.1. Benchmark Test

We'll benchmark the AI anomaly detection component against a rule-based baseline using precision, recall, F1-score, false-positive rate, and MTTD (mean time to detect). We'll inject anomaly scenarios in a controlled test environment.

3.4.2. Guideline Conformity Analysis

The final implementation will be assessed against the CIS Kubernetes Benchmark and CNCF Security Whitepaper recommendations. We'll report findings as a conformity scorecard in the final research report.

3.5. Framework Triangulation

Table 3.1 shows how we're using different research strategies to answer each sub-question.

Table 3.1: DOT Framework triangulation: research methods per sub-question.

Sub-question	Primary methods
SQ1 — Limitations of current approach	Problem analysis (Field), Literature study (Library)
SQ2 — AI/ML technique selection	Literature study (Library), Available product analysis (Library)
SQ3 — Cilium vs. traditional networking	Literature study (Library), Benchmark test (Showroom)
SQ4 — Integration architecture	IT architecture sketching (Library), Prototyping (Lab)
SQ5 — Validation	System test (Lab), Security test (Lab), Non-functional test (Lab), Guideline conformity (Showroom)

3.6. Additional Frameworks

Beyond the DOT framework, we're also using these methodological frameworks:

- **TOGAF ADM (Architecture Development Method):** The architecture design phase follows the TOGAF ADM cycle, moving from Architecture Vision through Business, Application, and

Technology Architecture to the Opportunities and Solutions phase.

- **ArchiMate 3.2:** We're using this as the modeling notation for all architecture views, ensuring interoperability and a standardized vocabulary.
- **GitOps (Flux CD):** All infrastructure-as-code artifacts are managed in a Git repository and automatically reconciled to the cluster, ensuring reproducibility of the proof-of-concept environment.

4

Non-Functional Requirements

The following non-functional requirements (NFRs) define constraints on how the integrated solution should perform and behave. They're based on enterprise infrastructure best practices, industry standards (CIS Kubernetes Benchmark, NIST SP 800-207), and the project context.

4.1. Scalability

- **NFR-SC-01:** The monitoring stack must work smoothly on clusters with up to 100 worker nodes and 500 running pods without needing manual reconfiguration.
- **NFR-SC-02:** The AI inference pipeline must scale horizontally — throughput should increase linearly when we add more inference replicas.
- **NFR-SC-03:** Cilium must operate at wire speed on 10 Gbps network interfaces without dropping packets under normal cluster load.

4.2. Performance

- **NFR-PE-01:** The extra CPU overhead from Cilium (eBPF programs) must not exceed 5% of total cluster CPU capacity compared to a Flannel baseline under the same workloads.
- **NFR-PE-02:** The extra memory footprint of the Cilium agent must not exceed 200 MiB per node.
- **NFR-PE-03:** End-to-end monitoring latency (from when a metric is generated to when an alert is sent) must be under 60 seconds for critical severity alerts.
- **NFR-PE-04:** AI model inference latency for a single prediction must be below 500 ms at the 99th percentile.

4.3. Security

- **NFR-SE-01:** All communication between services in the cluster must be controlled by explicit CiliumNetworkPolicy resources. We'll enforce a default-deny approach for both incoming and outgoing traffic.
- **NFR-SE-02:** Mutual TLS (mTLS) must be automatically enforced between all workloads using Cilium's WireGuard or IPsec encryption, without requiring any application code changes.
- **NFR-SE-03:** The solution must pass the applicable controls of the CIS Kubernetes Benchmark v1.9.
- **NFR-SE-04:** All container images used in the monitoring and security stack must be scanned for known vulnerabilities before deployment. We won't deploy images with critical CVEs.
- **NFR-SE-05:** Secrets (API keys, certificates, credentials) must not be stored in plaintext. They'll be managed via Kubernetes Secrets with RBAC access controls or an external secrets manager.

4.4. Reliability

- **NFR-RE-01:** The monitoring stack (Prometheus, Grafana, AlertManager) must be deployed in high-availability mode with at least two replicas per component and anti-affinity rules to prevent them from running on the same node.
- **NFR-RE-02:** We'll retain metric data for at least 30 days, with long-term storage backed by persistent volumes.
- **NFR-RE-03:** The Cilium agent must recover automatically from pod restarts without manual intervention, re-establishing all network policies within 30 seconds.
- **NFR-RE-04:** The AI anomaly detection service must be resilient to losing a single input data stream. It should keep producing predictions using available data and send a degraded-mode alert.

4.5. Observability

- **NFR-OB-01:** Full Layer 3 through Layer 7 network flow visibility must be available via Hubble for all traffic within the cluster and going out of the cluster.
- **NFR-OB-02:** The solution must support the three pillars of observability: metrics (Prometheus), logs (Loki), and distributed traces (OpenTelemetry/Jaeger), with the ability to correlate across all three.
- **NFR-OB-03:** Grafana dashboards must provide at minimum: cluster-level resource usage, per-namespace network flow heatmaps, anomaly detection score time series, and counts of active network policy violations.

4.6. Maintainability and Portability

- **NFR-MA-01:** All components must be deployed exclusively via Helm charts stored in a version-controlled Git repository. No manual `kubectl apply` steps should be needed after initial cluster setup.
- **NFR-MA-02:** The GitOps reconciliation loop (Flux CD) must automatically bring the cluster to the desired state defined in Git within 5 minutes of a commit.
- **NFR-MA-03:** The solution must be validated on at least two different Kubernetes distributions: bare-metal (kubeadm) and a managed cloud service (AKS or EKS).

5

Planning, Timeline, and Expected Challenges

5.1. Project Timeline

The remaining project period runs from 12 March 2026 to 28 June 2026. Working two days per week (09:00–16:00), this gives us roughly 15 working weeks. Table 5.1 breaks down each phase with concrete date ranges.

Table 5.1: Project timeline.

Phase	Dates	Activities	Deliverable
Project Plan	12 Mar	Finalize and submit plan of approach; supervisor review and approval	**This document**
Library Research	19 Mar – 2 Apr	Systematic literature study (AI/ML anomaly detection, eBPF, Cilium, zero-trust); available product analysis	Literature review
Architecture Design	2 Apr – 16 Apr	Problem analysis on reference cluster; domain modeling; ArchiMate architecture views (TOGAF ADM phases A–C)	Architecture document
Implementation — Cilium	16 Apr – 7 May	Install Cilium; configure CiliumNetworkPolicy (default-deny); enable WireGuard mTLS; deploy Hubble Relay and UI; validate Layer 3–7 policy enforcement	Cilium PoC
Implementation — AI Monitoring	7 May – 21 May	Deploy Prometheus + Loki + OpenTelemetry stack; build AI feature pipeline (Hubble → OTel Collector → feature store); train anomaly detection model; expose inference API	AI monitoring pipeline
Integration	21 May – 4 Jun	Integrate AI alerts with AlertManager; wire Hubble flow data into ML features; end-to-end smoke tests	Integrated PoC
Testing & Evaluation	4 Jun – 18 Jun	System tests; security tests (policy violation injection); non-functional tests (CPU/memory overhead); benchmark vs. threshold-based baseline	Test & benchmark report
Report Writing	18 Jun – 28 Jun	Draft final research report; peer review; incorporate feedback; final submission	**Final report**

5.2. Time Estimation

Available capacity: **2 days per week**, **09:00–16:00** (7 hours per day), from 12 March 2026 to 28 June 2026.

$$15 \text{ weeks} \times 2 \text{ days} \times 7 \text{ hours} = \textbf{210 hours total}$$

Table 5.2 shows the estimated effort distribution per phase.

Table 5.2: Estimated effort distribution.

Phase	Working days	Hours
Project Plan	1	7
Library Research	4	28
Architecture Design	4	28
Implementation — Cilium	5	35
Implementation — AI Monitoring	4	28
Integration	3	21
Testing & Evaluation	4	28
Report Writing	3	21
Buffer / contingency	2	14
Total	**30**	**210**

5.3. Expected Deliverables

We'll produce the following artifacts over the course of the project:

1. **Project Plan** (12 March) — this document.
2. **Literature Review** (2 April) — a structured review covering AI/ML anomaly detection techniques, Cilium/eBPF architecture, and relevant standards.
3. **Architecture Document** (16 April) — ArchiMate views (motivation, application, and technology layers) following the TOGAF ADM.
4. **Proof-of-Concept Implementation** (4 June) — all infrastructure-as-code (Helm charts, Flux manifests) and the AI pipeline source code, stored in a Git repository.
5. **Test & Benchmark Report** (18 June) — structured test results including CIS Benchmark conformity scorecard, security test outcomes, and performance benchmark comparisons.
6. **Final Research Report** (28 June) — a complete research report answering all research questions and providing actionable recommendations.

5.4. Expected Challenges

5.4.1. Training Data for AI Anomaly Detection

Training an unsupervised anomaly detection model requires a representative baseline of normal infrastructure behavior. Generating this baseline on a freshly set up test cluster means we'll have to train the model on synthetic or simulated workloads, which might not fully capture the diversity of production traffic patterns. Mitigation: we'll use an unsupervised technique (Isolation Forest) for the initial model, which doesn't need labeled anomaly data. We'll supplement it with synthetic anomaly injection for evaluation.

5.4.2. eBPF Kernel Version Compatibility

Some Cilium features (like WireGuard transparent encryption and socket-level load balancing) need a minimum Linux kernel version (5.10+). Bare-metal hardware running older kernels might not support all the features we're planning to use. Mitigation: we'll check kernel version requirements early during the Cilium implementation phase (starting 16 April). We'll document any feature gaps and use alternative mechanisms (IPsec) where WireGuard isn't available.

5.4.3. Integration Complexity

Bringing together different telemetry streams — Prometheus counter metrics, Loki log entries, OpenTelemetry spans, and Hubble flow records — into a unified feature vector for machine learning is tricky. Each data source has different cardinality, granularity, and latency characteristics. Mitigation: we'll use the OpenTelemetry Collector as a central pipeline with explicit transformation. We'll define a minimal feature set for the initial model and expand it iteratively.

5.4.4. Scope Creep

The combination of AI/ML and advanced networking covers a lot of ground. There's a risk of spending too much time on peripheral concerns (like model hyperparameter tuning or advanced Cilium enterprise features) instead of completing the core research. Mitigation: we'll strictly follow the timeline, use timeboxing for each implementation sprint, and discuss any scope changes with the supervisor before acting on them.

5.5. Supervisor Approval

This plan of approach needs to be reviewed and approved in writing by the supervising lecturer before research activities start on 19 March 2026.

Student:	**Supervisor:**
Bas Smit	Ederveen, Martin M.W.
Date: _____	Date: _____
Signature: _____	Signature: _____